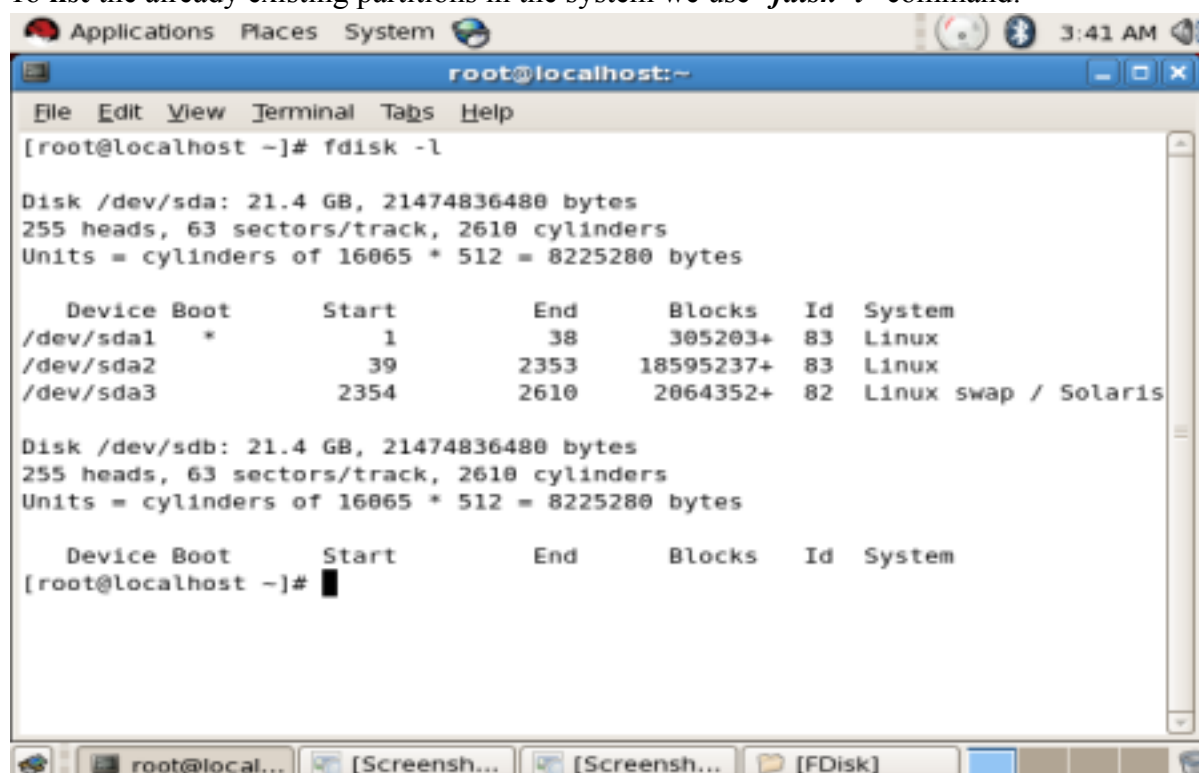


Aim: Demonstrate Disk Management commands for disk partitioning (create & delete partitions) in LINUX (Red Hat Enterprise Linux).

Theory : We know that Linux is a “multi-user” and “multi-tasking” system. We need to be in the “**root**” profile in order to **create/manage/delete** disk partitions. Enter RHEL through the “**root**” login in order to create partitions in your system. After you are in through “**root**” provide your “**username**” and “**password**” and log-in into the system:



To **list** the already existing partitions in the system we use “**fdisk -l**” command:

The image shows a terminal window titled "root@localhost:~". The window has a menu bar with "File", "Edit", "View", "Terminal", "Tabs", and "Help". The command "[root@localhost ~]# fdisk -l" has been entered. The output shows details for two disks: /dev/sda and /dev/sdb. Both are 21.4 GB. /dev/sda has three partitions: /dev/sda1 (Linux), /dev/sda2 (Linux), and /dev/sda3 (Linux swap / Solaris). /dev/sdb is unpartitioned. The terminal window is part of a desktop environment with a taskbar at the bottom showing icons for "root@local...", "[Screensh...", "[Screensh...", and "[FDisk]". The system clock in the top right corner shows "3:41 AM".

```
[root@localhost ~]# fdisk -l

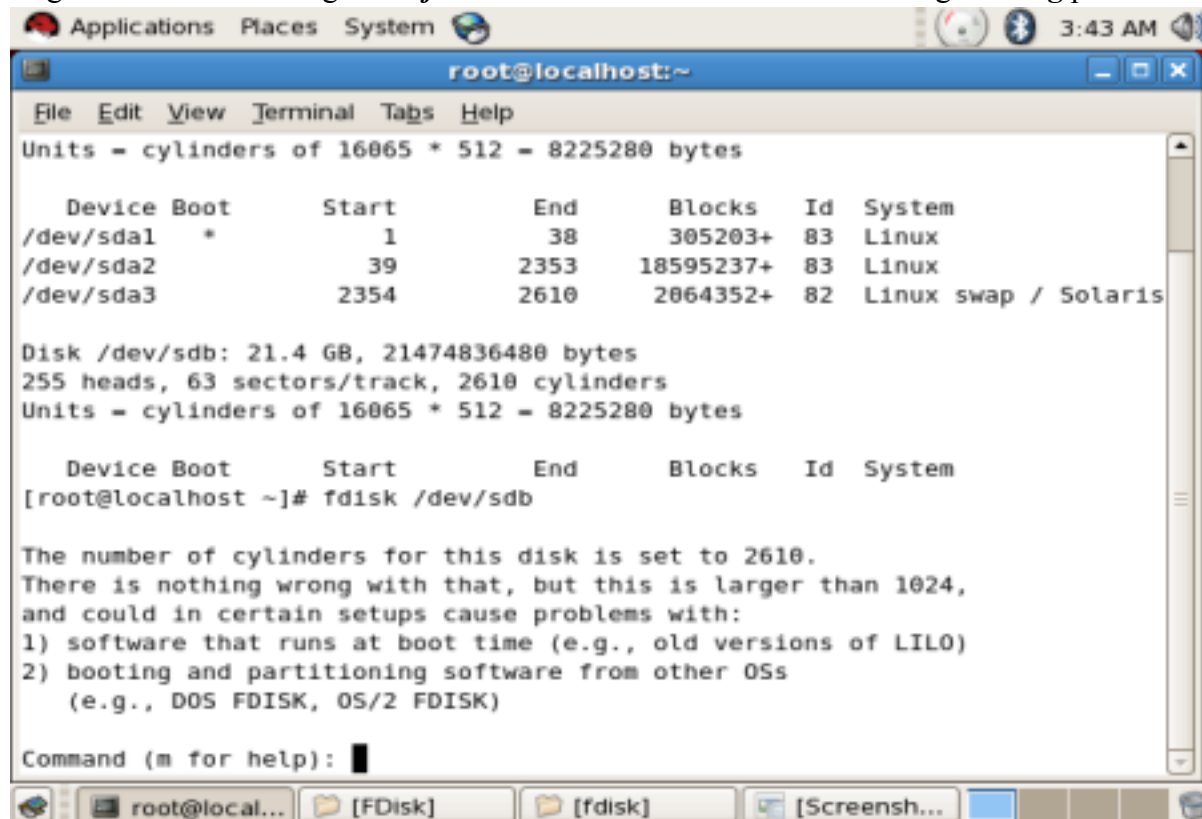
Disk /dev/sda: 21.4 GB, 21474836480 bytes
255 heads, 63 sectors/track, 2610 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes

   Device Boot      Start         End      Blocks    Id  System
/dev/sda1  *           1           38       305203+   83  Linux
/dev/sda2                39        2353     18595237+   83  Linux
/dev/sda3           2354        2610     2064352+   82  Linux swap / Solaris

Disk /dev/sdb: 21.4 GB, 21474836480 bytes
255 heads, 63 sectors/track, 2610 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes

   Device Boot      Start         End      Blocks    Id  System
[root@localhost ~]#
```

To get into the new storage use “*fdisk /dev/sdb*” which can be seen during **booting** process:



```
root@localhost:~  
File Edit View Terminal Tabs Help  
Units = cylinders of 16065 * 512 = 8225280 bytes  

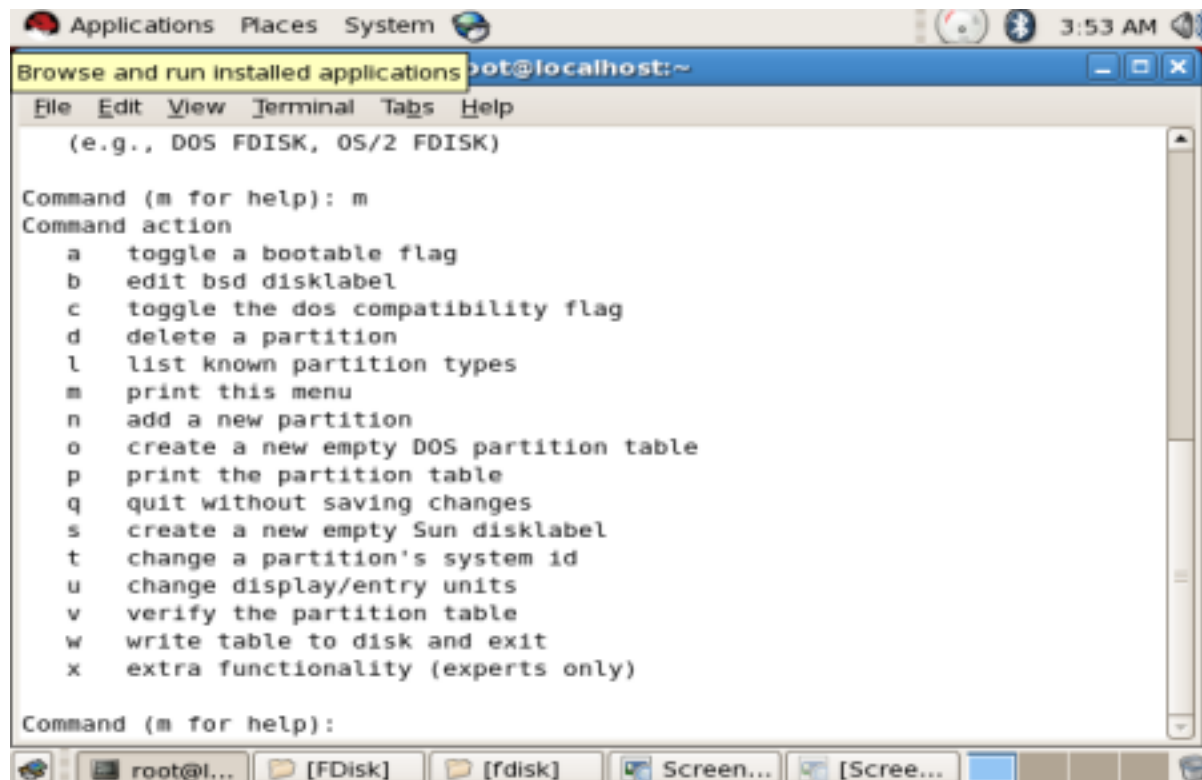

| Device    | Boot | Start | End  | Blocks    | Id | System               |
|-----------|------|-------|------|-----------|----|----------------------|
| /dev/sda1 | *    | 1     | 38   | 305203+   | 83 | Linux                |
| /dev/sda2 |      | 39    | 2353 | 18595237+ | 83 | Linux                |
| /dev/sda3 |      | 2354  | 2610 | 2064352+  | 82 | Linux swap / Solaris |

  
Disk /dev/sdb: 21.4 GB, 21474836480 bytes  
255 heads, 63 sectors/track, 2610 cylinders  
Units = cylinders of 16065 * 512 = 8225280 bytes  


| Device | Boot | Start | End | Blocks | Id | System |
|--------|------|-------|-----|--------|----|--------|
|--------|------|-------|-----|--------|----|--------|

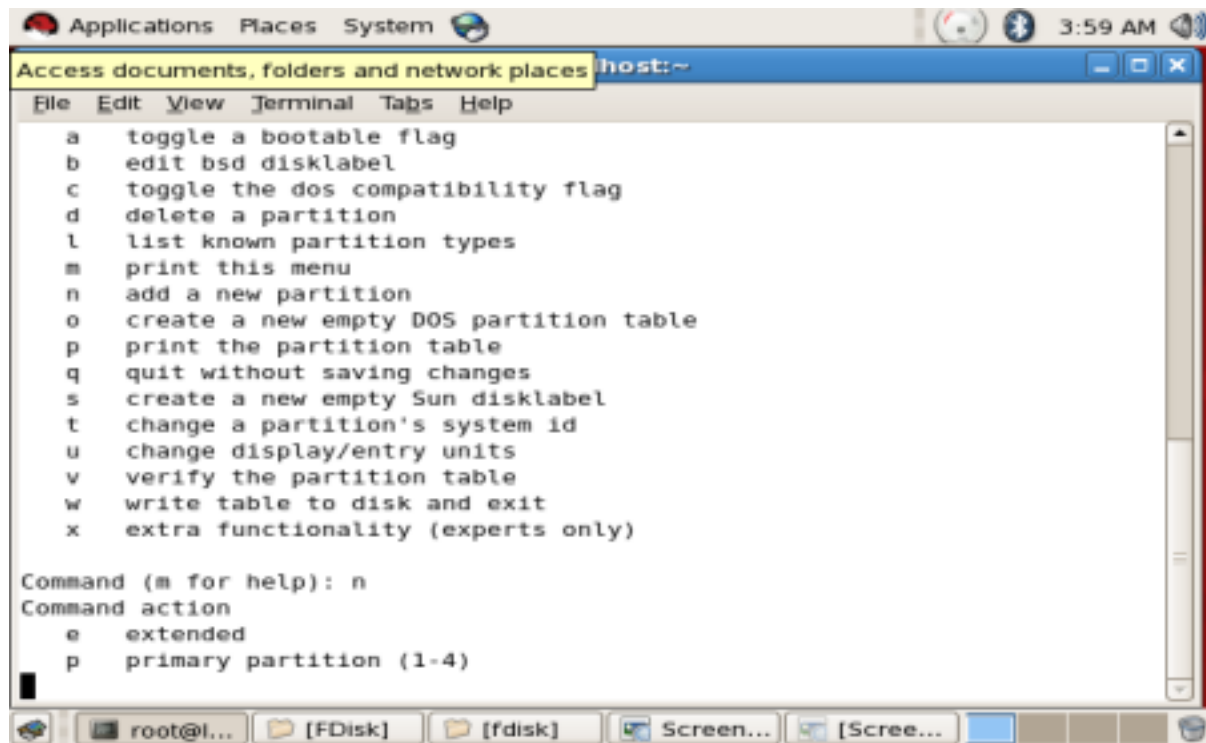
  
[root@localhost ~]# fdisk /dev/sdb  
  
The number of cylinders for this disk is set to 2610.  
There is nothing wrong with that, but this is larger than 1024,  
and could in certain setups cause problems with:  
1) software that runs at boot time (e.g., old versions of LILO)  
2) booting and partitioning software from other OSs  
   (e.g., DOS FDISK, OS/2 FDISK)  
  
Command (m for help):
```

In order to know all the commands associated with disk, you can use the “*m*” command which displays all the associated commands in “*fdisk*”.



```
root@localhost:~  
File Edit View Terminal Tabs Help  
(e.g., DOS FDISK, OS/2 FDISK)  
  
Command (m for help): m  
Command action  
a toggle a bootable flag  
b edit bsd disklabel  
c toggle the dos compatibility flag  
d delete a partition  
l list known partition types  
m print this menu  
n add a new partition  
o create a new empty DOS partition table  
p print the partition table  
q quit without saving changes  
s create a new empty Sun disklabel  
t change a partition's system id  
u change display/entry units  
v verify the partition table  
w write table to disk and exit  
x extra functionality (experts only)  
  
Command (m for help):
```

We have checked that we use “*-l*” in order to check for the known partition types. Now, in order to create a new partition we use “*n*”.

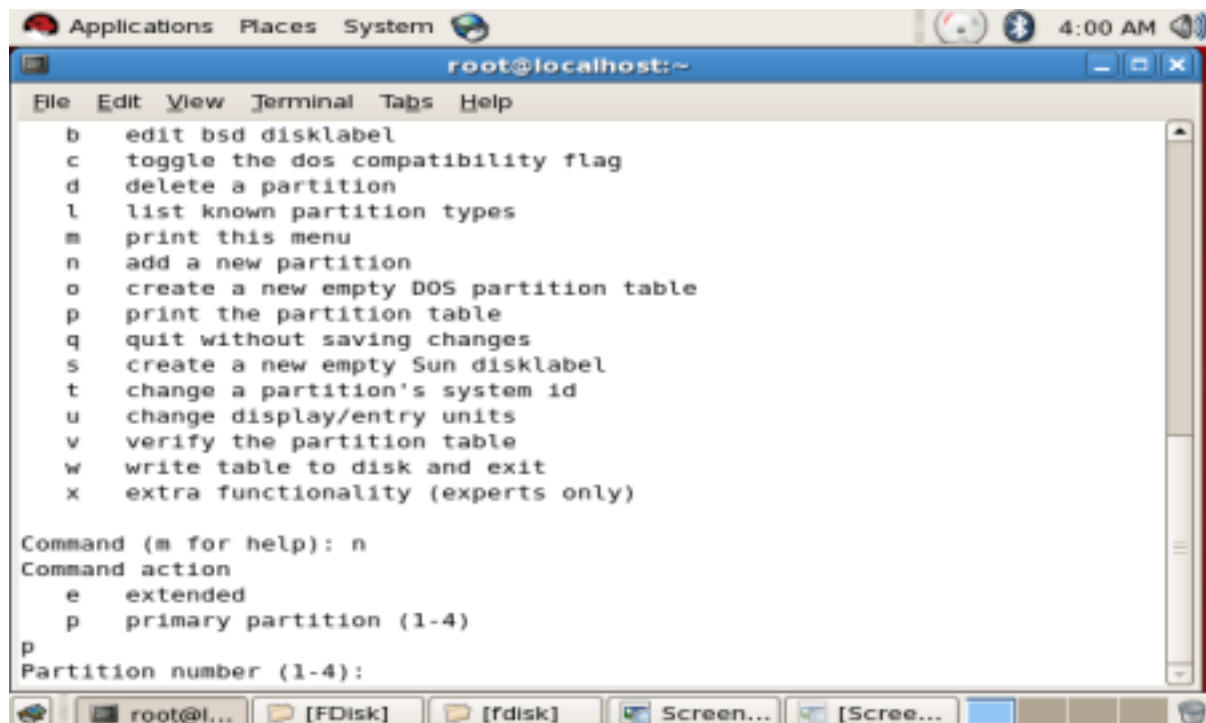
A screenshot of a terminal window titled "Access documents, folders and network places host:~". The window has a menu bar with "File", "Edit", "View", "Terminal", "Tabs", and "Help". The terminal displays the fdisk utility menu with options: a (toggle bootable flag), b (edit bsd disklabel), c (toggle dos compatibility flag), d (delete partition), l (list partition types), m (print menu), n (add new partition), o (create new empty DOS partition table), p (print partition table), q (quit without saving), s (create new empty Sun disklabel), t (change system id), u (change display/entry units), v (verify partition table), w (write table to disk and exit), and x (extra functionality). Below the menu, the user has entered "n" and the prompt "Command (m for help): n" is shown. The next prompt is "Command action", with options "e" for extended and "p" for primary partition (1-4). The user has entered "p".

```
Access documents, folders and network places host:~
File Edit View Terminal Tabs Help

a  toggle a bootable flag
b  edit bsd disklabel
c  toggle the dos compatibility flag
d  delete a partition
l  list known partition types
m  print this menu
n  add a new partition
o  create a new empty DOS partition table
p  print the partition table
q  quit without saving changes
s  create a new empty Sun disklabel
t  change a partition's system id
u  change display/entry units
v  verify the partition table
w  write table to disk and exit
x  extra functionality (experts only)

Command (m for help): n
Command action
  e  extended
  p  primary partition (1-4)
```

After using the “n” for new partition we use “p” in order to create a “primary partition”.

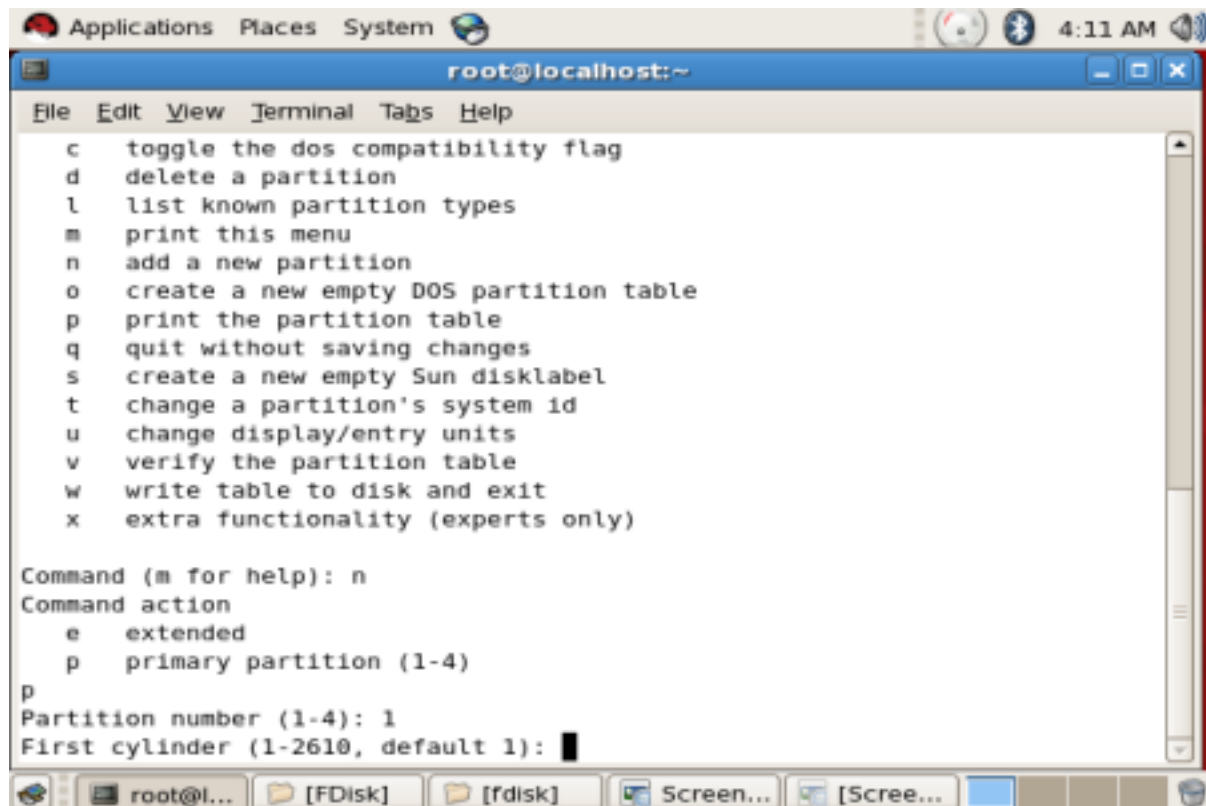
A screenshot of a terminal window titled "root@localhost:~". The window has a menu bar with "File", "Edit", "View", "Terminal", "Tabs", and "Help". The terminal displays the fdisk utility menu, which is identical to the one in the first screenshot. Below the menu, the user has entered "n" and the prompt "Command (m for help): n" is shown. The next prompt is "Command action", with options "e" for extended and "p" for primary partition (1-4). The user has entered "p". The next prompt is "Partition number (1-4):".

```
root@localhost:~
File Edit View Terminal Tabs Help

b  edit bsd disklabel
c  toggle the dos compatibility flag
d  delete a partition
l  list known partition types
m  print this menu
n  add a new partition
o  create a new empty DOS partition table
p  print the partition table
q  quit without saving changes
s  create a new empty Sun disklabel
t  change a partition's system id
u  change display/entry units
v  verify the partition table
w  write table to disk and exit
x  extra functionality (experts only)

Command (m for help): n
Command action
  e  extended
  p  primary partition (1-4)
p
Partition number (1-4):
```

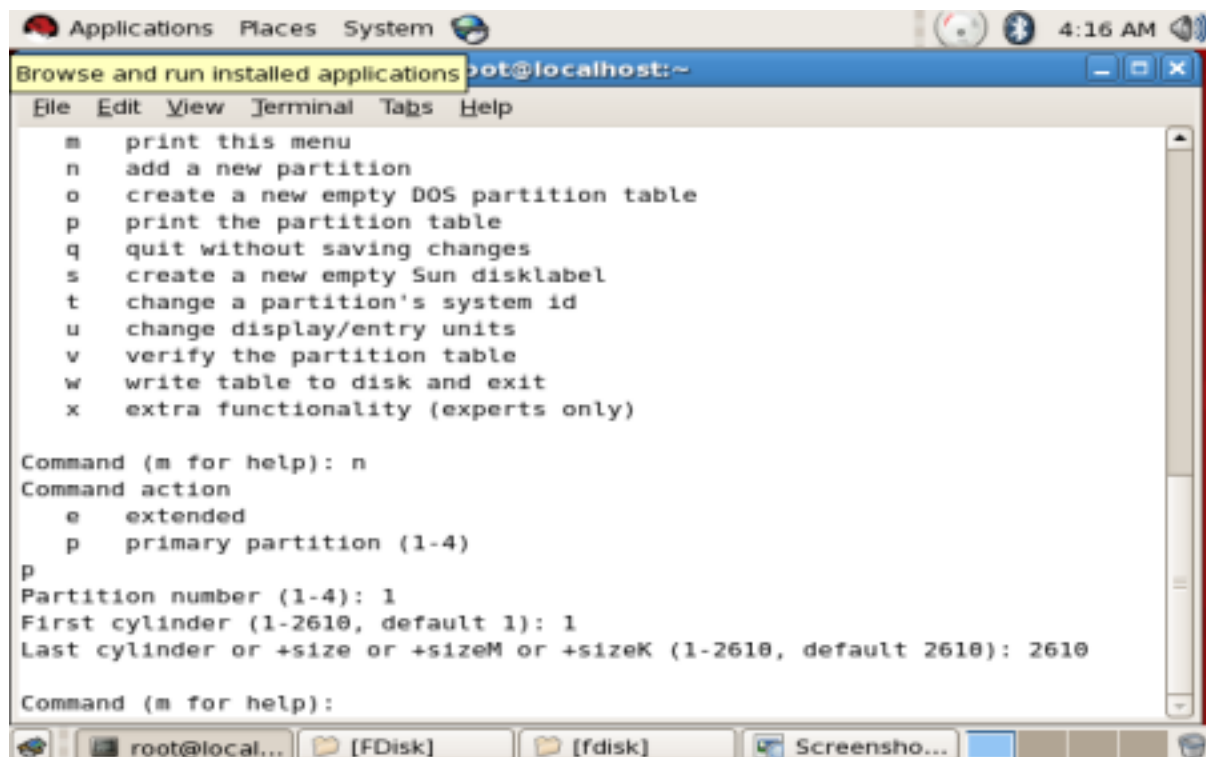
We can use the default section values, 1/2/3/4 to decide on the partition number which will be used by us. We will choose “1” generically, any other number can also be chosen.



The screenshot shows a terminal window titled 'root@localhost:~' with a menu of fdisk commands. The user has entered 'n' to create a new partition. The prompt 'Command (m for help): n' is followed by 'Command action' and a list of options: 'e' for extended and 'p' for primary partition (1-4). The user has entered 'p'. The prompt 'Partition number (1-4): 1' is followed by 'First cylinder (1-2610, default 1):' with a cursor at the end.

```
root@localhost:~  
File Edit View Terminal Tabs Help  
c toggle the dos compatibility flag  
d delete a partition  
l list known partition types  
m print this menu  
n add a new partition  
o create a new empty DOS partition table  
p print the partition table  
q quit without saving changes  
s create a new empty Sun disklabel  
t change a partition's system id  
u change display/entry units  
v verify the partition table  
w write table to disk and exit  
x extra functionality (experts only)  
  
Command (m for help): n  
Command action  
e extended  
p primary partition (1-4)  
p  
Partition number (1-4): 1  
First cylinder (1-2610, default 1):
```

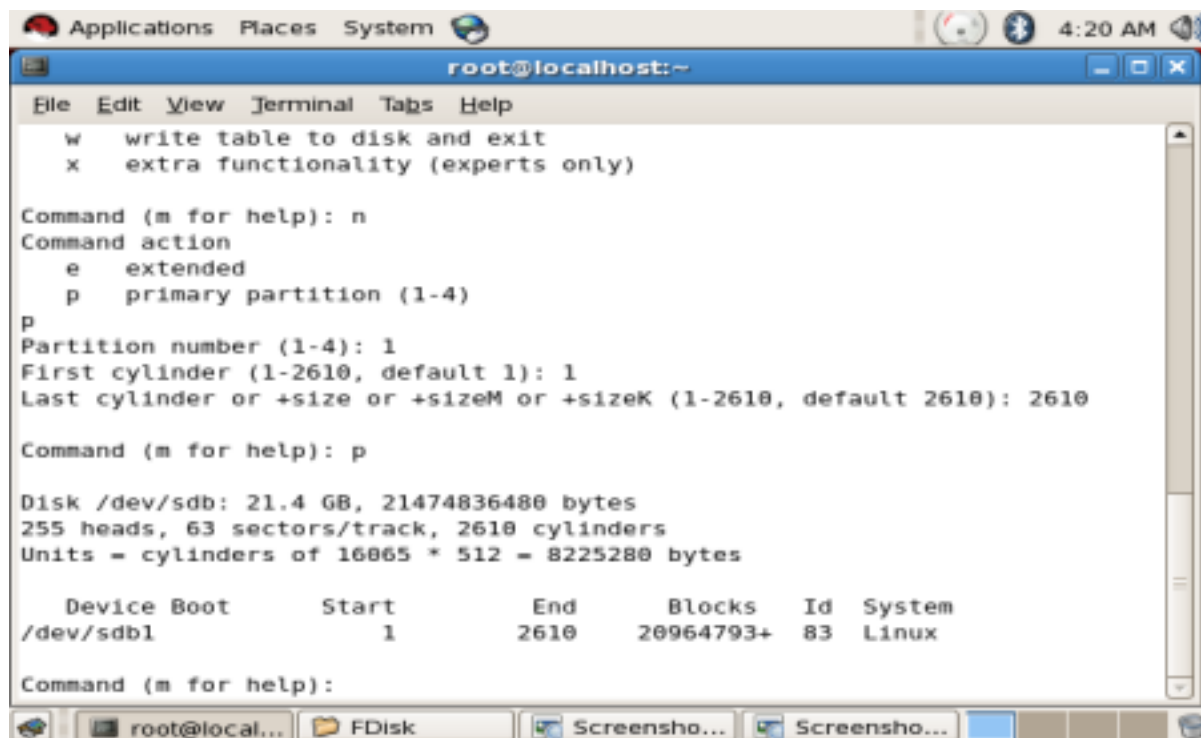
We are allocating the **first cylinder at partition 1** and also mentioning the default position at last, 2610 so that “**Partition space 1**” can be created. Generically while creating partitions we have to look at the prompts for space being allocated to us and specify it accordingly.



The screenshot shows the same terminal window as before, but now the user has entered '2610' for the last cylinder. The prompt 'Last cylinder or +size or +sizeM or +sizeK (1-2610, default 2610): 2610' is followed by 'Command (m for help):'. The window title has changed to 'Browse and run installed applications root@localhost:~'.

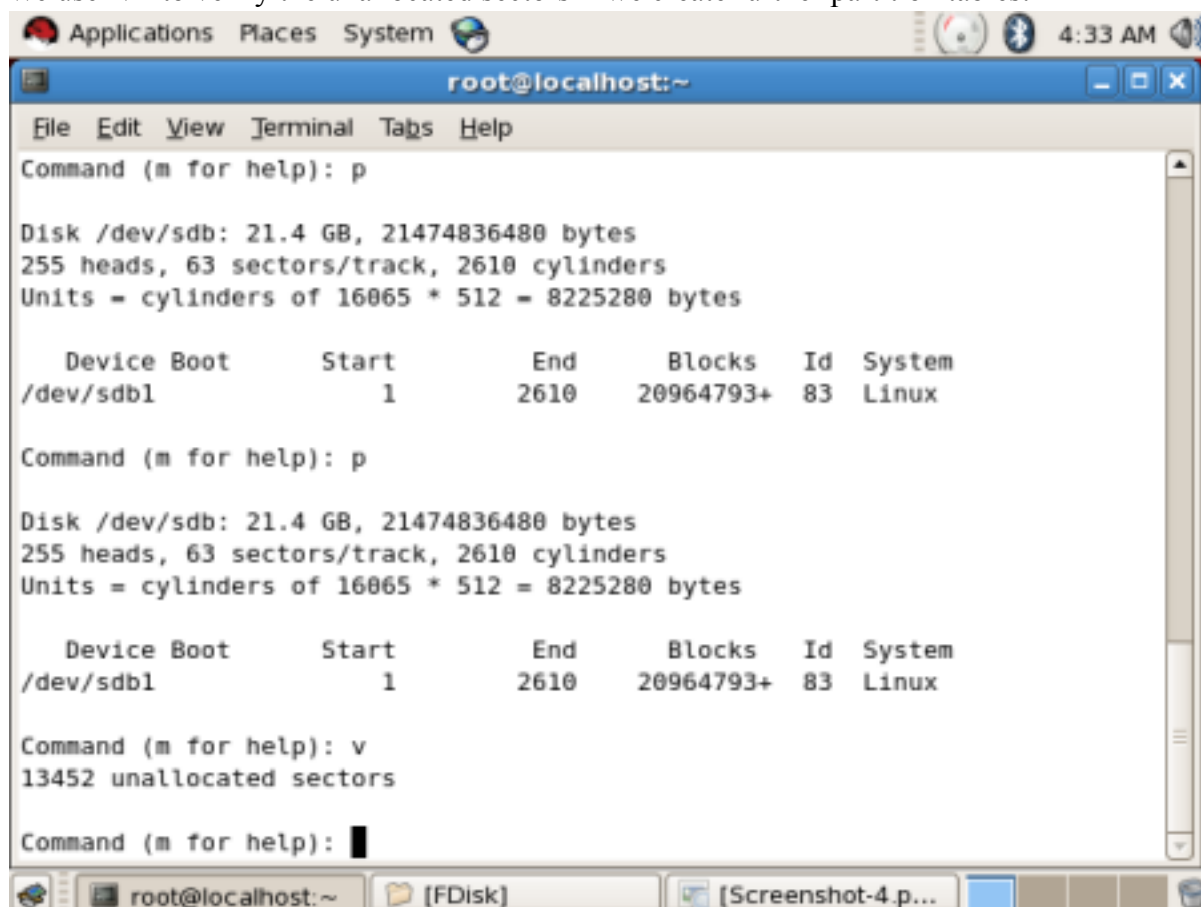
```
root@localhost:~  
File Edit View Terminal Tabs Help  
m print this menu  
n add a new partition  
o create a new empty DOS partition table  
p print the partition table  
q quit without saving changes  
s create a new empty Sun disklabel  
t change a partition's system id  
u change display/entry units  
v verify the partition table  
w write table to disk and exit  
x extra functionality (experts only)  
  
Command (m for help): n  
Command action  
e extended  
p primary partition (1-4)  
p  
Partition number (1-4): 1  
First cylinder (1-2610, default 1): 1  
Last cylinder or +size or +sizeM or +sizeK (1-2610, default 2610): 2610  
Command (m for help):
```

After creating the 1st partition according to commands in disk partitioning we use “**p**” to print the details of the partition created:



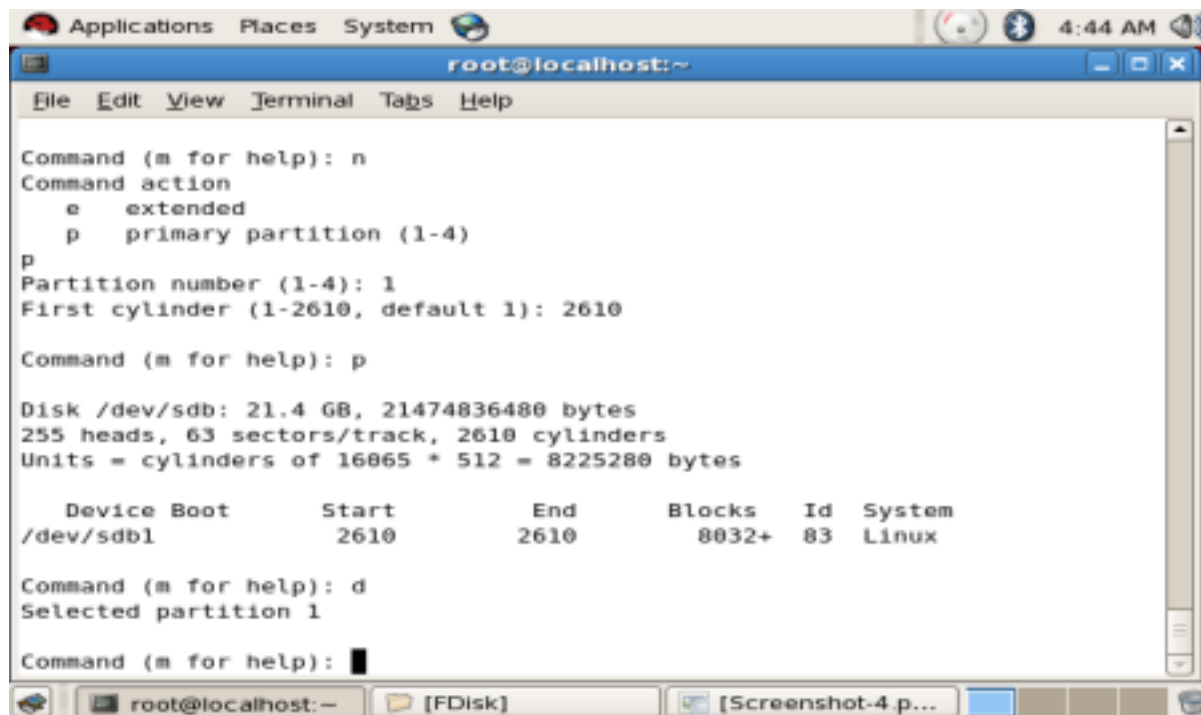
```
root@localhost:~  
File Edit View Terminal Tabs Help  
w write table to disk and exit  
x extra functionality (experts only)  
  
Command (m for help): n  
Command action  
e extended  
p primary partition (1-4)  
p  
Partition number (1-4): 1  
First cylinder (1-2610, default 1): 1  
Last cylinder or +size or +sizeM or +sizeK (1-2610, default 2610): 2610  
  
Command (m for help): p  
  
Disk /dev/sdb: 21.4 GB, 21474836480 bytes  
255 heads, 63 sectors/track, 2610 cylinders  
Units = cylinders of 16065 * 512 = 8225280 bytes  
  
   Device Boot      Start         End      Blocks   Id  System  
/dev/sdb1            1         2610    20964793+   83   Linux  
  
Command (m for help):
```

We use “v” to verify the unallocated sectors if we create further partition tables:



```
root@localhost:~  
File Edit View Terminal Tabs Help  
Command (m for help): p  
  
Disk /dev/sdb: 21.4 GB, 21474836480 bytes  
255 heads, 63 sectors/track, 2610 cylinders  
Units = cylinders of 16065 * 512 = 8225280 bytes  
  
   Device Boot      Start         End      Blocks   Id  System  
/dev/sdb1            1         2610    20964793+   83   Linux  
  
Command (m for help): p  
  
Disk /dev/sdb: 21.4 GB, 21474836480 bytes  
255 heads, 63 sectors/track, 2610 cylinders  
Units = cylinders of 16065 * 512 = 8225280 bytes  
  
   Device Boot      Start         End      Blocks   Id  System  
/dev/sdb1            1         2610    20964793+   83   Linux  
  
Command (m for help): v  
13452 unallocated sectors  
  
Command (m for help):
```

In order to **delete the partitions** we use the “d” command whose output is as follows:

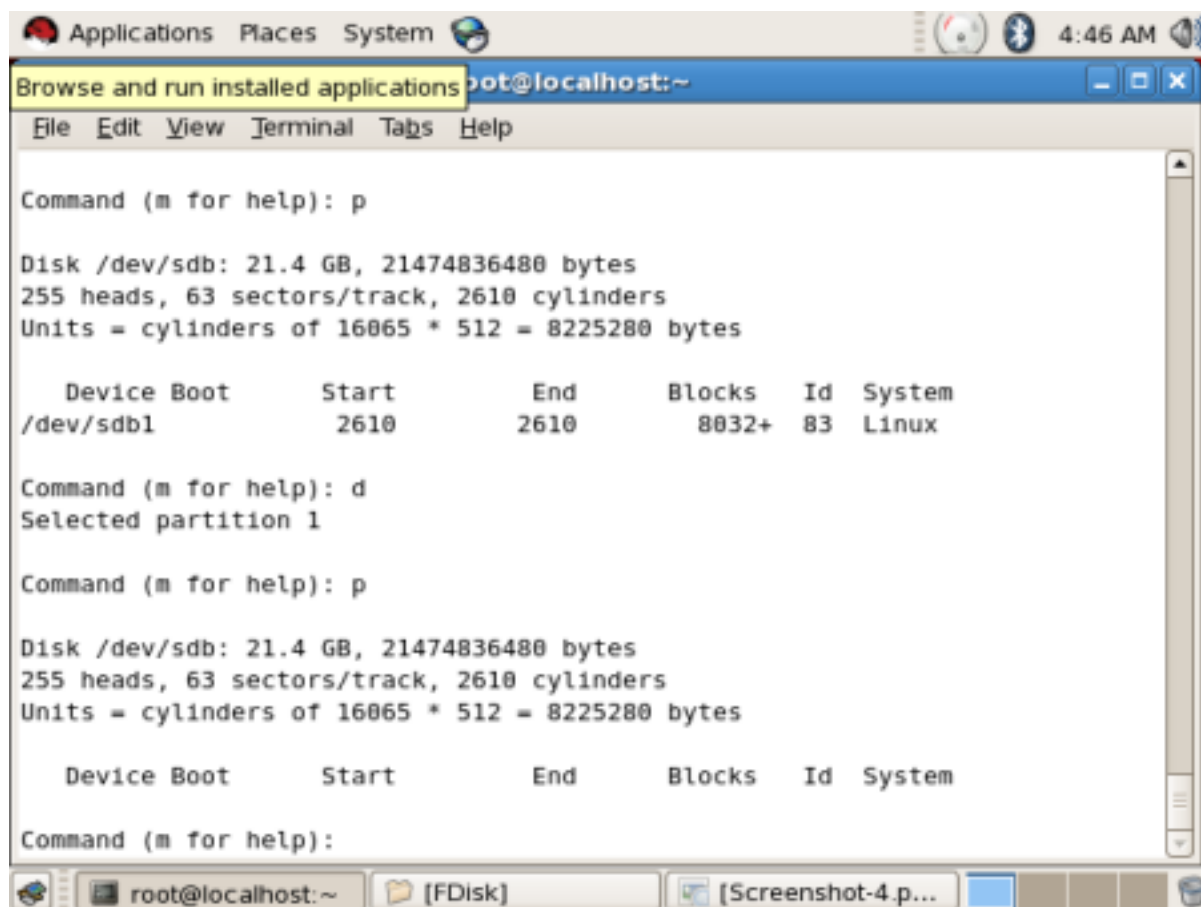


```
root@localhost:~  
File Edit View Terminal Tabs Help  
Command (m for help): n  
Command action  
  e   extended  
  p   primary partition (1-4)  
p  
Partition number (1-4): 1  
First cylinder (1-2610, default 1): 2610  
Command (m for help): p  
Disk /dev/sdb: 21.4 GB, 21474836480 bytes  
255 heads, 63 sectors/track, 2610 cylinders  
Units = cylinders of 16065 * 512 = 8225280 bytes  


| Device    | Boot | Start | End  | Blocks | Id | System |
|-----------|------|-------|------|--------|----|--------|
| /dev/sdb1 |      | 2610  | 2610 | 8032+  | 83 | Linux  |

  
Command (m for help): d  
Selected partition 1  
Command (m for help):
```

In order to check if the partition has been deleted we use the “*p*” command which is used to print all the partitions which exist in the system.



```
root@localhost:~  
File Edit View Terminal Tabs Help  
Command (m for help): p  
Disk /dev/sdb: 21.4 GB, 21474836480 bytes  
255 heads, 63 sectors/track, 2610 cylinders  
Units = cylinders of 16065 * 512 = 8225280 bytes  


| Device    | Boot | Start | End  | Blocks | Id | System |
|-----------|------|-------|------|--------|----|--------|
| /dev/sdb1 |      | 2610  | 2610 | 8032+  | 83 | Linux  |

  
Command (m for help): d  
Selected partition 1  
Command (m for help): p  
Disk /dev/sdb: 21.4 GB, 21474836480 bytes  
255 heads, 63 sectors/track, 2610 cylinders  
Units = cylinders of 16065 * 512 = 8225280 bytes  


| Device | Boot | Start | End | Blocks | Id | System |
|--------|------|-------|-----|--------|----|--------|
|--------|------|-------|-----|--------|----|--------|

  
Command (m for help):
```


Aim: Build a shell script in order to show the various system configuration information.

Theory: Shell Variables

The variable is a place holder for storing the data. The value of the variable can be changed during the program execution. The value assigned could be a number, text, filename, device, or any other type of data. A variable is nothing more than a pointer to the actual data. The shell enables you to create, assign, and delete variables. Shell variables can be used at the command line and/or used within shell programs. While similar to other programming language variables, shell variables do have some different characteristics such as:

- shell variables do not need to be declared
- all shell variables are of type string

There are two types of variables:

- User defined variables (UDV)
- System Defined variables (SDV)

How to assign value to a variable?

Syntax: variable-name=value 'value' is assigned to given 'variable name' and Value must be on right side = sign.

System variables

Created and maintained by Linux/Unix itself. This type of variable is defined in CAPITAL LETTERS. **These variables are generally set using the “export” command.** Some of the commonly required variables are as follows.

PS1 //Command prompt

HOME //your home directory

PWD //your present working directory

LOGNAME //your login name

USER // same as login name

HISTSIZE // no. of history commands to be stored in history file

HOSTNAME //your host name

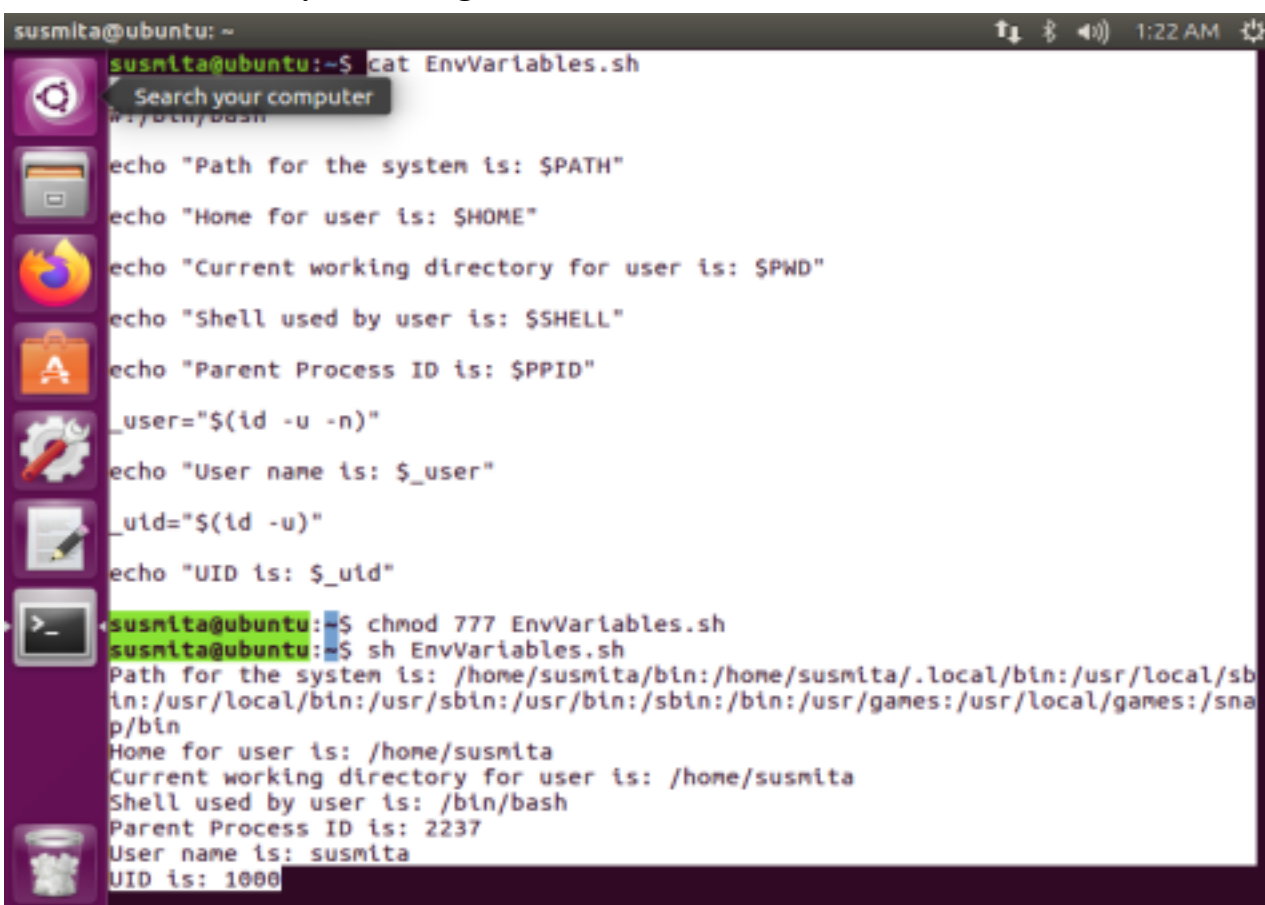
SHELL //your login shell

PPID //Process id of parent process

For system allocated variables like username/ UID etc. we cannot directly access them like the environment variables studied earlier as these variables are created by the system on the user's behalf. Irrespective of system defined or user defined variables, they are retrieved using the \$ symbol.

Following is the demonstration for a shell script with the problem statement to **show the**

various system configuration information.



```
susmita@ubuntu: ~  
susmita@ubuntu:~$ cat EnvVariables.sh  
echo "Path for the system is: $PATH"  
echo "Home for user is: $HOME"  
echo "Current working directory for user is: $PWD"  
echo "Shell used by user is: $SHELL"  
echo "Parent Process ID is: $PPID"  
_user="$(id -u -n)"  
echo "User name is: $_user"  
_uid="$(id -u)"  
echo "UID is: $_uid"  
susmita@ubuntu:~$ chmod 777 EnvVariables.sh  
susmita@ubuntu:~$ sh EnvVariables.sh  
Path for the system is: /home/susmita/bin:/home/susmita/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin  
Home for user is: /home/susmita  
Current working directory for user is: /home/susmita  
Shell used by user is: /bin/bash  
Parent Process ID is: 2237  
User name is: susmita  
UID is: 1000
```

Aim: Build a shell script that accepts the hostname and IP address as command-line arguments and adds them to the “/etc/hosts” file.

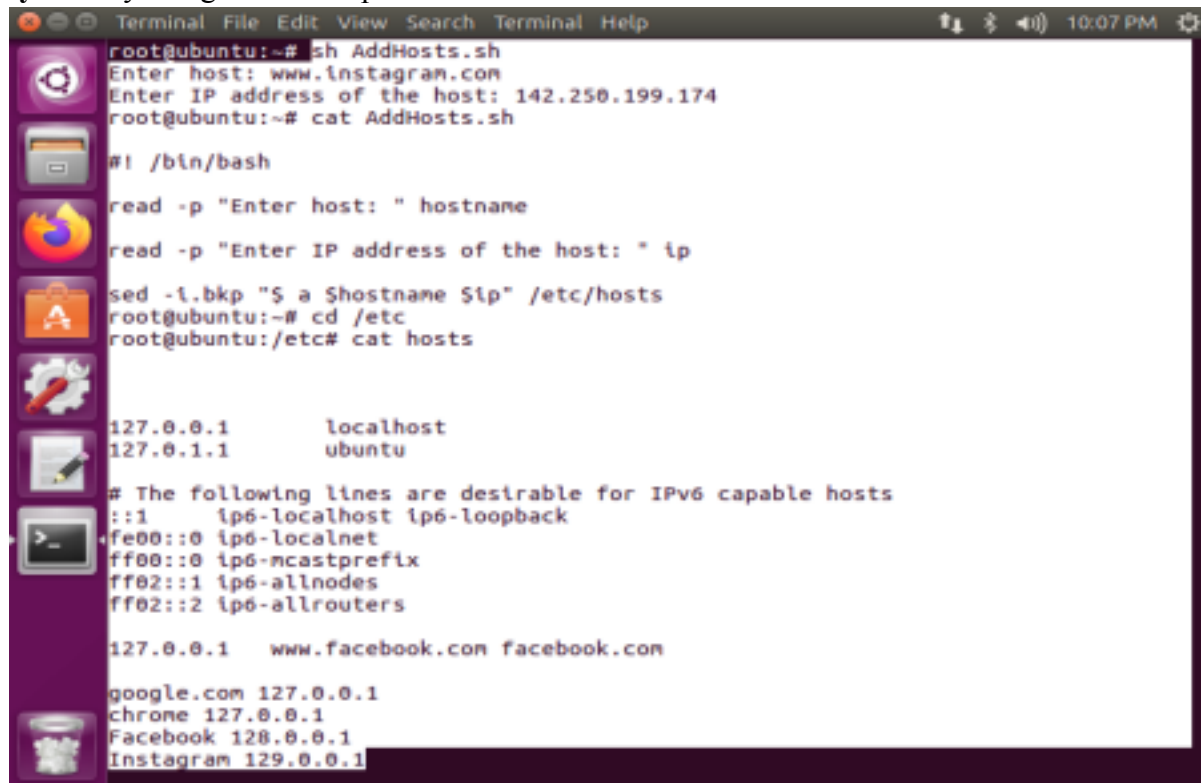
Theory: The “hosts” file is used to map domain names (hostnames) to IP addresses. It is a plain text file used by all operating systems including, Linux, Windows, and macOS. The hosts file has priority over DNS (Domain Name Server). When you type in the domain name of a web site you want to visit, the domain name must be translated into its corresponding IP Address. The operating system first checks its “hosts” file for the corresponding domain, and if there is no entry for the domain, it will query the configured DNS servers to resolve the specified domain name. This affects only the computer on which the change is made, rather than how the domain is resolved worldwide.

Using the “hosts” file to map a domain to an IP address is particularly useful when you want to test your website without changing the domain DNS settings. For example, you are migrating your website to a new server, and you want to verify whether it is fully functional before pointing the domain to the new server. The hosts file can also be used to block websites on your computer.

Within this context of utilities of the “hosts” file we will explore in this experiment how we can edit the “hosts” file using a shell script. The underlying objective is to understand how to add hosts to our system, and also how to ensure the safety & security of our Linux system by filtering only those

hosts/ websites which are authenticated by the particular administrator of the system. This measure can be further extended to different types of users, groups as discussed in earlier experiments.

In this program we check the utility of shell scripts w.r.t **to adding the hosts directly in our Linux system** by using a shell script.



```
root@ubuntu:~# sh AddHosts.sh
Enter host: www.instagram.com
Enter IP address of the host: 142.250.199.174
root@ubuntu:~# cat AddHosts.sh

#!/bin/bash

read -p "Enter host: " hostname
read -p "Enter IP address of the host: " ip

sed -i.bkp "$ a $hostname $ip" /etc/hosts
root@ubuntu:~# cd /etc
root@ubuntu:/etc# cat hosts

127.0.0.1    localhost
127.0.1.1    ubuntu

# The following lines are desirable for IPv6 capable hosts
::1         ip6-localhost ip6-loopback
fe00::0     ip6-localnet
ff00::0     ip6-mcastprefix
ff02::1     ip6-allnodes
ff02::2     ip6-allrouters

127.0.0.1    www.facebook.com facebook.com

google.com 127.0.0.1
chrome 127.0.0.1
Facebook 128.0.0.1
Instagram 129.0.0.1
```

Aim: Assess remote network applications viz. “Remote server access/administration” with SSH (Secure Shell) & PuTTY (where TTY stands for terminal teletype).

Theory: PuTTY is a free, light-weight, open-source terminal emulator. It is a serial port network file transfer application. It supports several network protocols, including SCP, SSH, Telnet, rlogin, and raw socket connection. PuTTY is an SSH and telnet client, developed originally by Simon Tatham for the Windows platform; (also available for installation on Linux platforms).

An SSH client is a software program which uses the secure shell protocol to connect to a remote computer. You will find PuTTY useful if you want to access an account on a Unix or other multi user system from a PC (for example your own or one in an internet cafe). PuTTY is an alternative to telnet clients. Its primary advantage is that SSH provides a secure, encrypted connection to the remote system. It's also small and self-contained and can be carried around on a floppy disk.

Use of PuTTY: In a multi-user operating system like Unix, the interface is generally of command line type, just like the command prompt or MS-DOS. As such the user needs to type in the command in the command line program to get anything processed by the system. Generally, these commands can quickly be run over a network from a different computer on a different location (client) and the response is transferred over the network to the client.

The arrangement mentioned above is made possible with the help of network protocols like SSH, Telnet, Rlogin, etc. Interestingly, users can give commands to multiple computers simultaneously.

SSH (Secure Shell) protocol is a cryptographic network protocol that allows you to access an internet server while encrypting any information sent to that server. Some of the other protocols include Telnet, Rlogin only if either you are connected to a Unix system or you have a login account on a web server (a shell account). PuTTY is one such application that enables this kind of transfer.

How to Manage a Session in PuTTY

It is the preliminary panel where you get to specify specific options to open a session.

- The **Host Name (or IP Address)** bar is where a user will input the name or IP address of the server they want to connect.
- **Connection type** of radio buttons allows users to choose from the kind of network they are planning to connect.
- The Port bar is the section that is filled automatically on selecting the type of connection. However, if you choose the **Raw** type, the bar stays blank and requires the user to enter the **port** manually.
- Upon selecting Serial as the connection type, the **Host Name and Port bars** will be replaced by **Serial Line and Speed**.

The “**Load, save or delete a stored session**” section is to set some connection setting without having to type all the details again when needed.

- Once you save it, it can just select on the saved session and click on Load. The saved settings will appear on their respective boxes in the configuration panel.
- The panel permits to modify a saved session by first loading a session, editing everything you want to change and then clicking on “save” button.
- Users have the option to delete a session as well.

The **Close Window On Exit** option helps in deciding whether the PuTTY terminal will close as soon as the session ends or restarts the session on the termination.

What is Logging in PuTTY?

This configuration panel saves the log files of your PuTTY sessions which can be used for debugging and analysis purposes. Users can choose the type of data you want to log in this window.

What is Terminal in PuTTY?

The section has a variety of options to decide how the texts in the window should appear. Whether you want the text to come in the next line as soon as it reaches the right edge of the window or you want to interpret the cursor position.

Configure Keyboard setting in PuTTY.

With the option, users can modify the behavior of ‘backspace,’ ‘home’ and ‘end’ keys, and several other keys to coordinate with the server settings.

What is Bell in PuTTY?

It lets PuTTY make an alert sound as and when you want it to function.

Manage Data in PuTTY.

The auto-login option dismisses the need to type the username every time. It can also specify the terminal needs using this panel.

A proxy setting in PuTTY

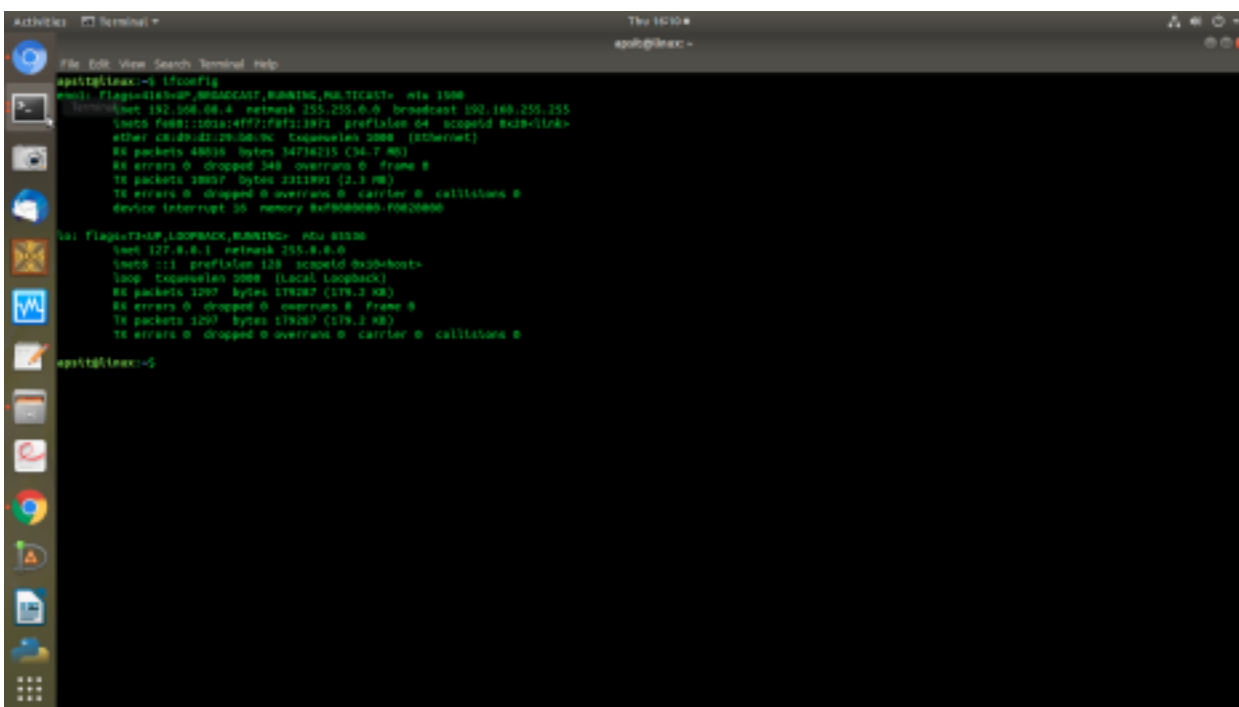
Permits configuration of various types of proxies used while making the network connections.

What are Telnet, Rlogin, and SSH in PuTTY?

These panels allow configuration of session-related options like changing the mode for negotiation between the server and client, allowing an automated form of login on the server, etc.

Accessing remote networks using SSH & Putty:

The following terminal with the **IP address (192.168.88.4)** is our Server machine which we try to connect to through the **PuTTY** (installed Windows/Linux enabled terminal). We obtain this IP using the command **ifconfig** in our Linux system.

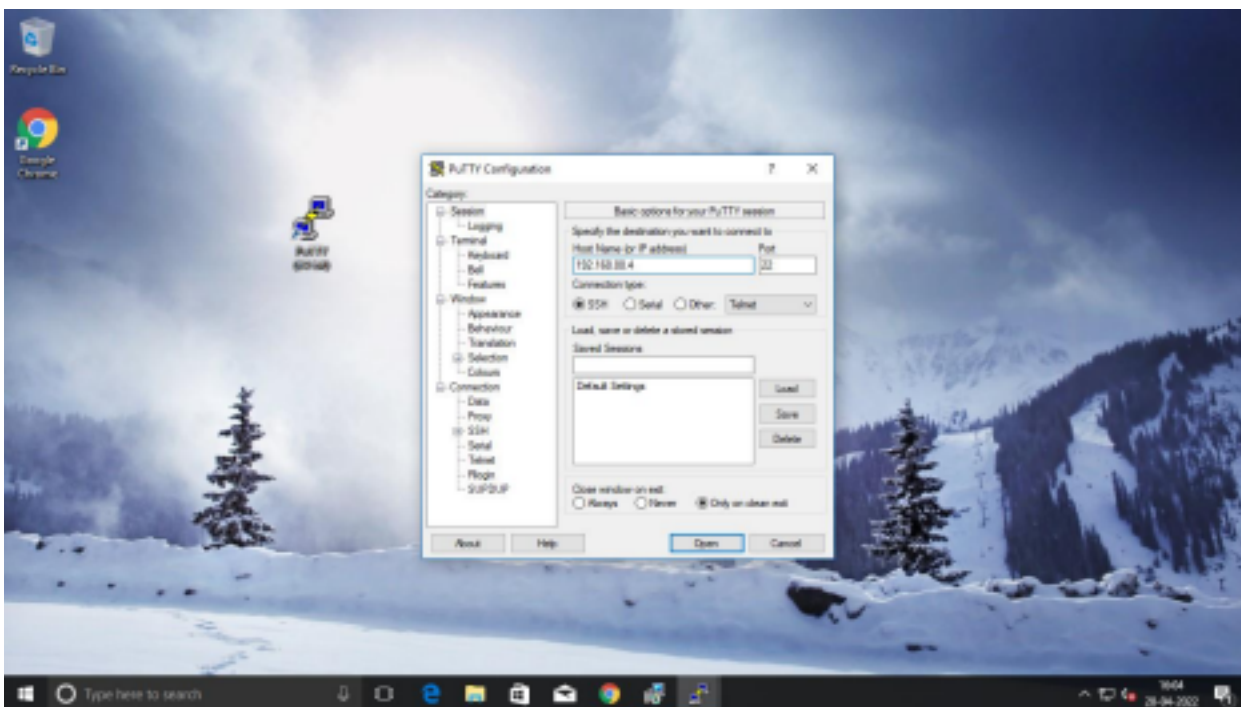


```
apalt@linux:~$ ifconfig
enx1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.88.4 netmask 255.255.0.0 broadcast 192.168.255.255
    ether 08:00:27:12:34:56 txqueuelen 1000 (Ethernet)
    RX packets 48036 bytes 34734215 (34.7 MB)
    RX errors 0 dropped 348 overruns 0 frame 0
    TX packets 38037 bytes 2211991 (2.1 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 35 memory 8a700000-8a702000

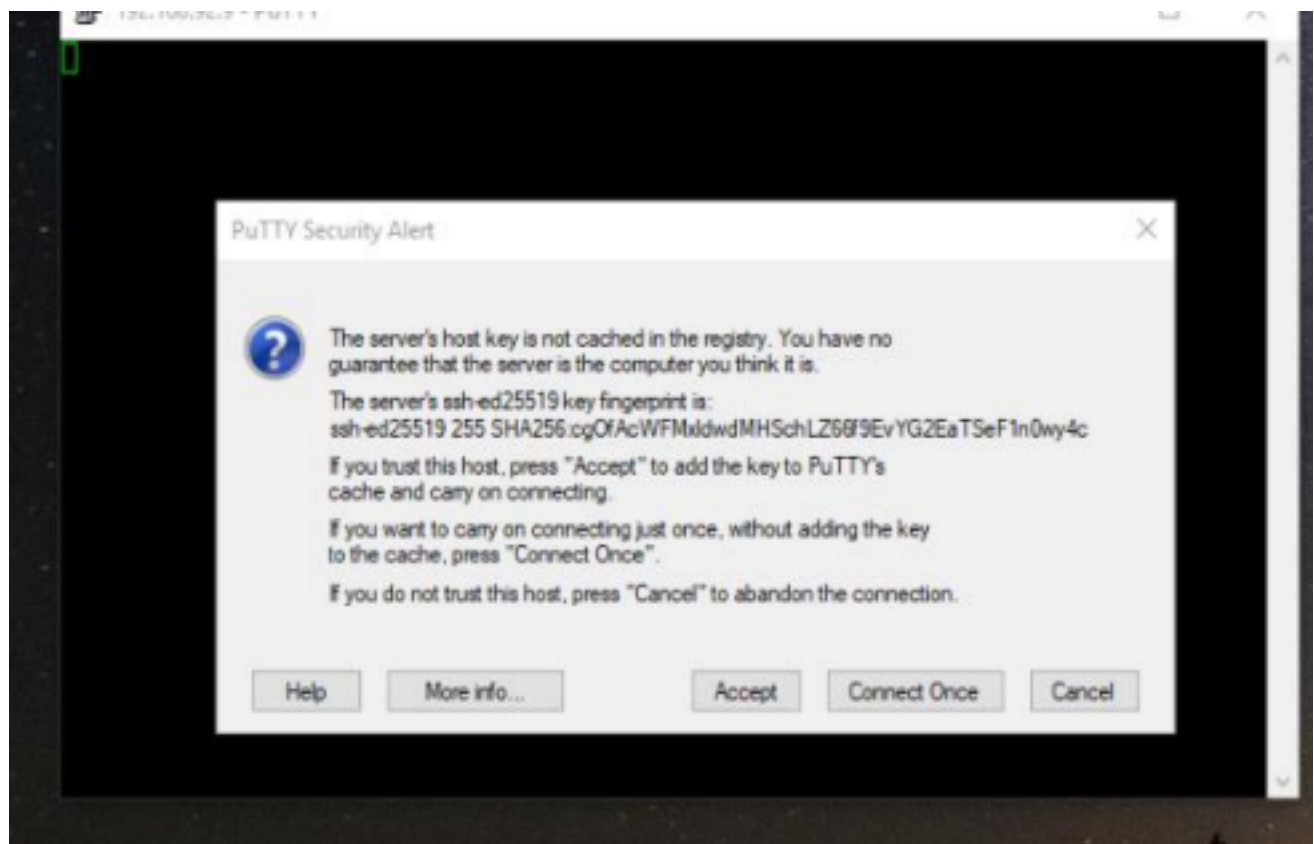
lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1297 bytes 179187 (179.1 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1297 bytes 179187 (179.1 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

apalt@linux:~$
```

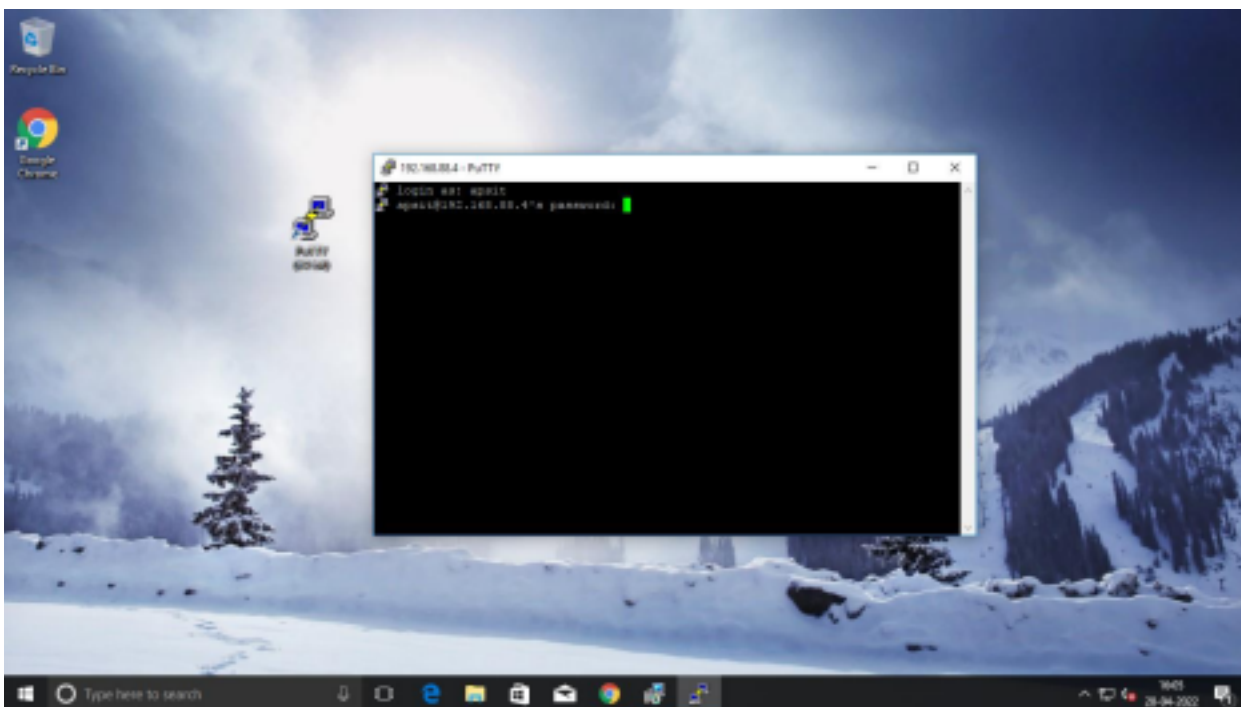
We will download the Putty application on our Windows Client machine. Post this, in order to connect to the “Server” we login through the “**Putty Configuration**” window. On terminal (Client side) we will provide the IP address of the machine we want to connect to as seen.



In the following screen, we can observe that the **“PuTTY Security Alert”** screen prompts the user to enter choices between: **“Accept”**, **“Connect Once”**, **“Cancel”**. We will choose the **“Accept”** option so that we can make a trusted connection with the **Server**.



Login with the credentials of **“login as”** and **“password”** of the system for PuTTY:



Once we login the immediate screen visible is informative with details like the release version of Ubuntu, the no. of packages updated and also others like Documentation, Management & Support info.

```
apsit@linux:~$
apsit@linux:~$ login as: apsit
apsit@192.168.88.4's password:
Access denied
apsit@192.168.88.4's password:
Welcome to Ubuntu 18.04.4 LTS (GNU/Linux 4.15.0-167-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

 * Super-optimized for small spaces - read how we shrank the memory
   footprint of MicroK8s to make it the smallest full K8s around.
   https://ubuntu.com/blog/microk8s-memory-optimisation

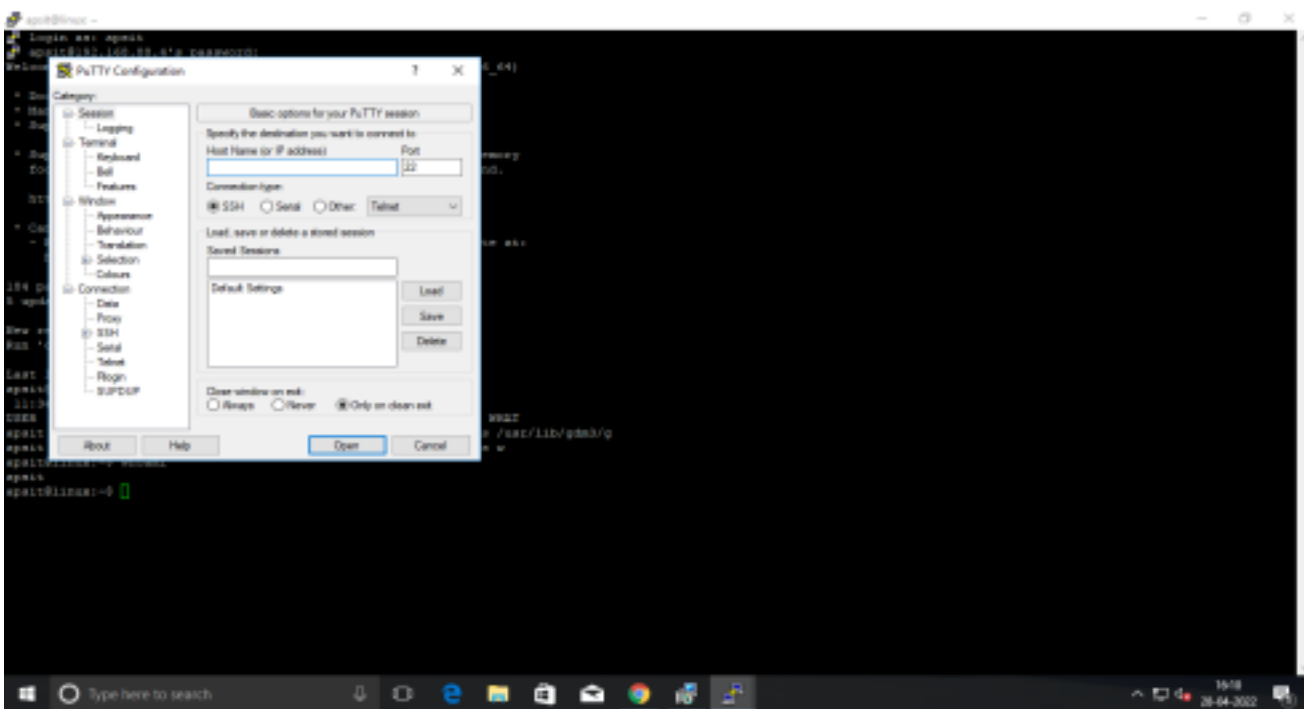
 * Canonical Livepatch is available for installation.
   - Reduce system reboots and improve kernel security. Activate at:
     https://ubuntu.com/livepatch

184 packages can be updated.
5 updates are security updates.

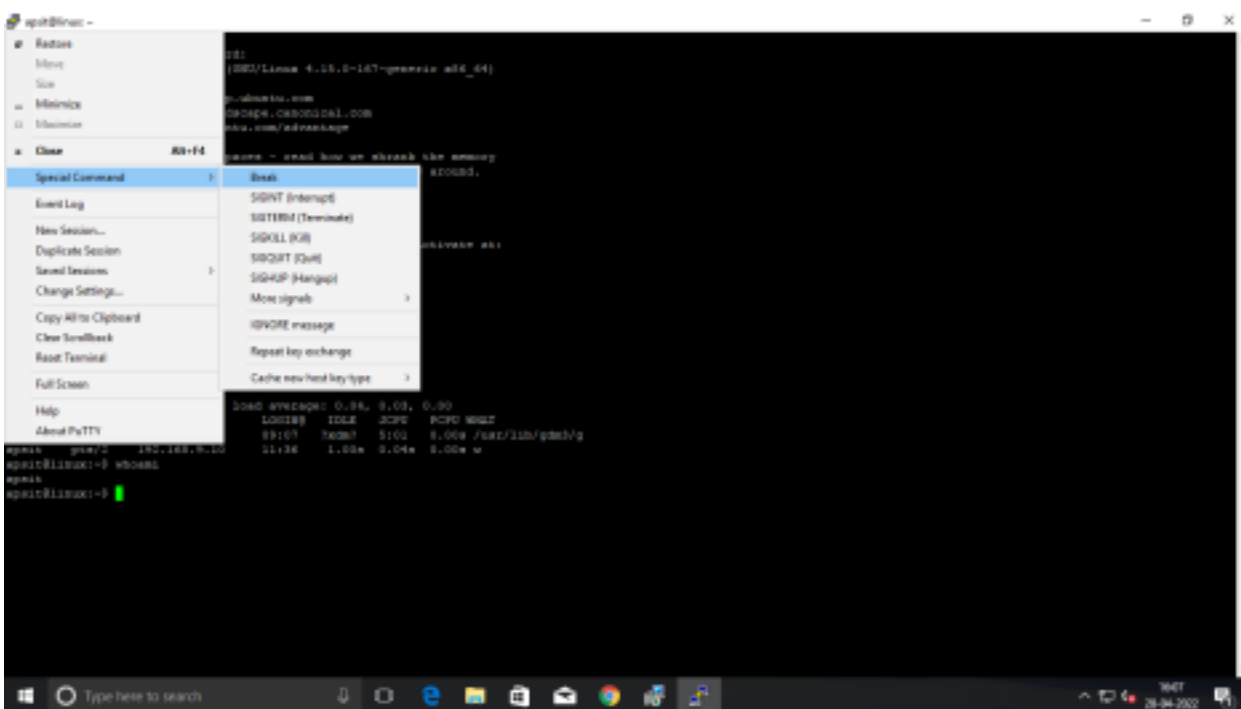
New release '20.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Thu Apr 28 12:37:18 2022 from 192.168.9.10
apsit@linux:~$
```

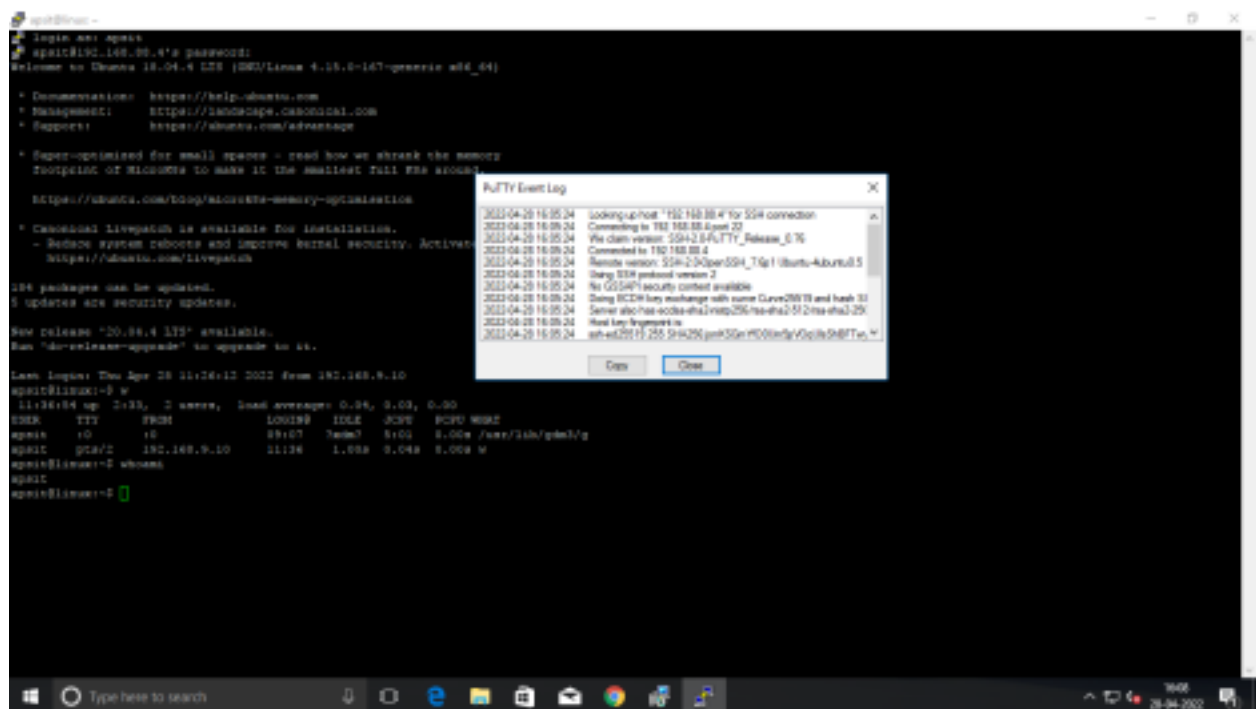
Whenever you create a “**New connection**” use the IP address of the machine you must connect to:



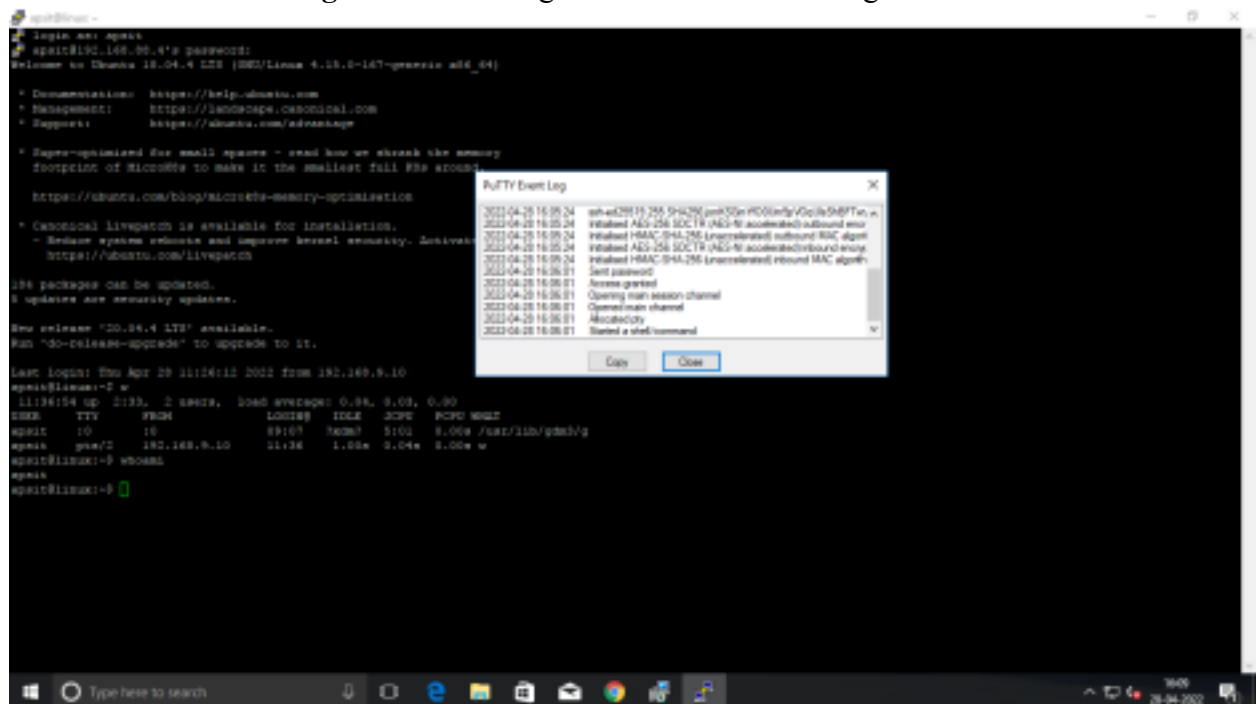
Special commands available (on Linux system) and that can be also be accessed through **remote login via PuTTY** are **SIGINT/SIGTERM/SIGKILL** to mention a few:



Software interrupts on Linux and Unix systems are made via signals. **There are many different Linux signals, but a few stand out and are important to understand and know: SIGINT, SIGTERM, and SIGKILL.** The basic Linux signals all have a number (1-30+). After a little while, a proficient Linux user will generally know one or more of these. For example, the SIGTERM signal matches with number 15, and signal 9 (SIGKILL) is likely the most known one as it allows one to forcefully terminate a process, a SIGTERM signal is used to send a warning signal to a process; however, the process/program may choose to ignore such a signal.



The “PuTTY Event Log” maintains a log of all the activities being executed on the machine.



For a different session with Server IP (192.168.7.43) we observe in the following screens that the same output is observable when we use the command “w” on the **Server and on Client machine**:

Aim: Configuration of NFS (Network File System) Server and transfer files to a Linux Client.

Theory: NFS (Network File System) is a distributed file system. An NFS server has one or more file systems that are mounted by NFS clients. These file systems are mounted by using the standard UNIX mount commands. NFS is the

standard method to share files between Linux and Unix-like systems.

NFS is a networking protocol for distributed file sharing. A file system defines the way files are stored and retrieved from storage devices, such as hard disk drives, solid-state drives and on tape drives across networks. It enables network administrators to share all or a portion of a file system on a networked server to make it accessible to remote computer users. NFS is one of the most widely used protocols for file servers. Cloud vendors also implement the NFS protocol for cloud storage, including Amazon Elastic File System, NFS file shares in Microsoft Azure and Google Cloud Filestore.

How does the Network File System work?

NFS is a client-server protocol. An NFS Server is a host that meets the following requirements:

- has NFS installed
- has at least one network connection for sharing NFS resources; and
- is configured to accept and respond to NFS requests over the network connection. An NFS Client is a host

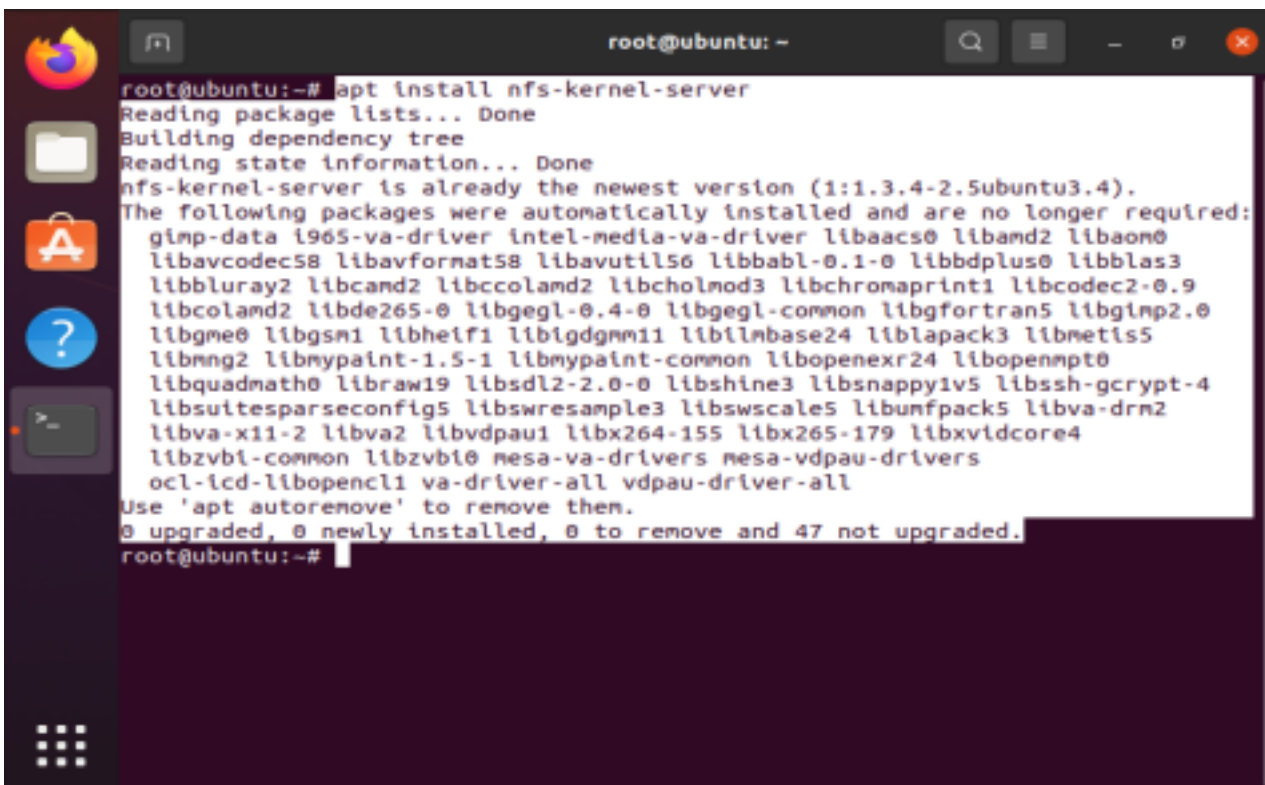
that meets the following requirements:

- has NFS client software installed
- has network connectivity to an NFS server
- is authorized to access resources on the NFS server; and
- is configured to send and receive NFS requests over the network connection.

NFS was initially conceived as a method for sharing file systems across workgroups using UNIX. It is still often used for ad hoc sharing of resources.

Installation of NFS Server

Install the package for “**nfs-kernel-server**” on your machine.

A terminal window titled 'root@ubuntu: ~' with standard Ubuntu window controls. The terminal shows the command 'apt install nfs-kernel-server' being executed. The output indicates that the package is already installed at the latest version (1:1.3.4-2.5ubuntu3.4) and lists 47 packages that are no longer required. The user is prompted to use 'apt autoremove' to remove them. The terminal ends with the prompt 'root@ubuntu:~#'.

```
root@ubuntu:~# apt install nfs-kernel-server
Reading package lists... Done
Building dependency tree
Reading state information... Done
nfs-kernel-server is already the newest version (1:1.3.4-2.5ubuntu3.4).
The following packages were automatically installed and are no longer required:
gimp-data i965-va-driver intel-media-va-driver libaac0 libamd2 libaom0
libavcodec58 libavformat58 libavutil56 libbabi-0.1-0 libbdplus0 libblas3
libbluray2 libcand2 libccolamd2 libcholmod3 libchromaprint1 libcodec2-0.9
libcolamd2 libde265-0 libgegl-0.4-0 libgegl-common libgfortran5 libgimp2.0
libgme0 libgsn1 libhelf1 libigdgmm1 libilmbase24 liblapack3 libmetis5
libmng2 libmypaint-1.5-1 libmypaint-common libopenexr24 libopenmpt0
libquadmath0 libraw19 libstdl2-2.0-0 libshine3 libsnappy1v5 libssh-gcrypt-4
libsuitesparseconfig5 libswresample3 libswscale5 libunfpack5 libva-drm2
libva-x11-2 libva2 libvdpau1 libx264-155 libx265-179 libxvidcore4
libzvti-common libzvti0 mesa-va-drivers mesa-vdpau-drivers
ocl-icd-libopencl1 va-driver-all vdpau-driver-all
Use 'apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 47 not upgraded.
root@ubuntu:~#
```

```
root@ubuntu: ~  
libsuitesparseconfig5 libswresample3 libswscale5 libunfpack5 libva-drm2  
libva-x11-2 libva2 libvdpau1 libx264-155 libx265-179 libxvidcore4  
libzvbi-common libzvbi0 mesa-va-drivers mesa-va-drivers  
ocl-icd-libopencl1 va-driver-all vdpau-driver-all  
Use 'apt autoremove' to remove them.  
0 upgraded, 0 newly installed, 0 to remove and 47 not upgraded.  
root@ubuntu:~#  
root@ubuntu:~#  
root@ubuntu:~#  
root@ubuntu:~# apt install nfs-common  
Reading package lists... Done  
Building dependency tree  
Reading state information... Done  
nfs-common is already the newest version (1:1.3.4-2.5ubuntu3.4).  
The following packages were automatically installed and are no longer required:  
gimp-data i965-va-driver intel-media-va-driver libaacs0 libamd2 libaom0  
libavcodec58 libavformat58 libavutil56 libbabi-0.1-0 libbdplus0 libblas3  
libbluray2 libcamd2 libccolamd2 libcholmod3 libchromaprint1 libcodec2-0.9  
libcolamd2 libde265-0 libgegl-0.4-0 libgegl-common libgfortran5 libgimp2.0  
libgme0 libgsm1 libheif1 libigdgmm1 libilmbase24 liblapack3 libmetis5  
libmng2 libmypaint-1.5-1 libmypaint-common libopenexr24 libopenmpt0  
libquadmath0 libraw19 libstdl2-2.0-0 libshine3 libsnappy1v5 libssh-gcrypt-4  
libsuitesparseconfig5 libswresample3 libswscale5 libunfpack5 libva-drm2  
libva-x11-2 libva2 libvdpau1 libx264-155 libx265-179 libxvidcore4  
libzvbi-common libzvbi0 mesa-va-drivers mesa-va-drivers  
ocl-icd-libopencl1 va-driver-all vdpau-driver-all  
Use 'apt autoremove' to remove them.  
0 upgraded, 0 newly installed, 0 to remove and 47 not upgraded.  
root@ubuntu:~#
```

Create the folder that needs to be shared,

```
root@ubuntu:~#  
root@ubuntu:~# cd /mnt  
root@ubuntu:/mnt#  
root@ubuntu:/mnt#  
root@ubuntu:/mnt# ls -lrt  
total 0  
root@ubuntu:/mnt#  
root@ubuntu:/mnt# mkdir nfsshare  
root@ubuntu:/mnt#  
root@ubuntu:/mnt# ls -lrt  
total 4  
drwxr-xr-x 2 root root 4096 Apr 27 07:35 nfsshare  
root@ubuntu:/mnt#
```

Provide permissions as below,

```
root@ubuntu:/mnt#  
root@ubuntu:/mnt# chown -R nobody:nogroup nfsshare  
root@ubuntu:/mnt#  
root@ubuntu:/mnt# ls -lrt  
total 4  
drwxr-xr-x 2 nobody nogroup 4096 Apr 27 07:35 nfsshare  
root@ubuntu:/mnt#  
root@ubuntu:/mnt#
```

```
root@ubuntu:/mnt#  
root@ubuntu:/mnt# chmod -R 777 nfsshare/  
root@ubuntu:/mnt#  
root@ubuntu:/mnt#  
root@ubuntu:/mnt# ls -lrt  
total 4  
drwxrwxrwx 2 nobody nogroup 4096 Apr 27 07:35 nfsshare  
root@ubuntu:/mnt#  
root@ubuntu:/mnt#
```

Edit “/etc/exports” with below entry with IP or (*) for all servers,

```
# /etc/exports: the access control list for filesystems which may be exported
# to NFS clients. See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_sub
tree_check)
#
# Example for NFSv4:
# /srv/nfs4 gss/krb5l(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes gss/krb5l(rw,sync,no_subtree_check)
/mnt/nfsshare 192.168.75.129(rw,sync,no_root_squash,insecure)
-
-
-
```

//add the IP address of

server with details of the file being shared

Check the status of NFS service after making these changes,

```
root@ubuntu:~#
root@ubuntu:~# systemctl status nfs-kernel-server
● nfs-server.service - NFS server and services
   Loaded: loaded (/lib/systemd/system/nfs-server.service; enabled; vendor p
   Active: active (exited) since Wed 2022-04-27 07:44:41 PDT; 34s ago
   Process: 6243 ExecStartPre=/usr/sbin/exportfs -r (code=exited, status=0/SU
   Process: 6244 ExecStart=/usr/sbin/rpc.nfsd $RPCNFSDARGS (code=exited, statu
   Main PID: 6244 (code=exited, status=0/SUCCESS)

Apr 27 07:44:40 ubuntu systemd[1]: Starting NFS server and services...
Apr 27 07:44:41 ubuntu systemd[1]: Finished NFS server and services.
```

Check the firewall

status,

```
root@ubuntu:~# ufw status
Status: inactive
root@ubuntu:~#
root@ubuntu:~#
```

Export the NFS folder and mount the shared folder,

```
root@ubuntu:/mnt#
root@ubuntu:/mnt# exportfs -rav
exportfs: /etc/exports [3]: Neither 'subtree_check' or 'no_subtree_check' speci
fied for export "*/mnt/nfsshare".
   Assuming default behaviour ('no_subtree_check').
   NOTE: this default has changed since nfs-utils version 1.0.x

exporting */mnt/nfsshare
root@ubuntu:/mnt#
root@ubuntu:/mnt#
root@ubuntu:/mnt# mount 192.168.75.129:/mnt/nfsshare /mnt/nfs_client_share
root@ubuntu:/mnt#
root@ubuntu:/mnt# ls -lrt
total 8
drwxrwxrwx 2 nobody nogroup 4096 Apr 27 07:35 nfsshare
drwxrwxrwx 2 nobody nogroup 4096 Apr 27 07:35 nfs_client_share
root@ubuntu:/mnt#
```

Folder acting as server, has a "a.txt" file

```
root@ubuntu:/mnt# cd nfsshare/
root@ubuntu:/mnt/nfsshare#
root@ubuntu:/mnt/nfsshare#
root@ubuntu:/mnt/nfsshare#
root@ubuntu:/mnt/nfsshare#
root@ubuntu:/mnt/nfsshare# ls -lrt
total 4
-rw-r--r-- 1 root root 28 Apr 27 08:06 a.txt
root@ubuntu:/mnt/nfsshare#
root@ubuntu:/mnt/nfsshare#
root@ubuntu:/mnt/nfsshare# cat a.txt
This file is shared on NFS.
root@ubuntu:/mnt/nfsshare#
root@ubuntu:/mnt/nfsshare#
```

Text file acting as a server has a.txt file and is also now visible via Client_share machine


```
root@ubuntu:/mnt#  
root@ubuntu:/mnt#  
root@ubuntu:/mnt# cd nfs_client_Share/  
root@ubuntu:/mnt/nfs_client_Share#  
root@ubuntu:/mnt/nfs_client_Share#  
root@ubuntu:/mnt/nfs_client_Share# ls -lrt  
total 4  
-rw-r--r-- 1 root root 28 Apr 27 08:06 a.txt  
root@ubuntu:/mnt/nfs_client_Share#  
root@ubuntu:/mnt/nfs_client_Share#  
root@ubuntu:/mnt/nfs_client_Share# cat a.txt  
This file is shared on NFS.
```